

AirPrice India

SIH26056 -- Complete Technical Documentation

Smart Hackathon India 2026 | Problem ID: SIH26056 | Tools, Architecture, Workflow, Challenges & Results | September 2026

This document provides a thorough explanation of every tool used in the prototype, the complete end-to-end data workflow, the real challenges the system faces at each stage and exactly how it tackles them, and the final outputs delivered. It is written so that any team member or external reviewer can understand the architecture without needing to read the source code.

1. What This Project Does and Why It Matters

India's Ministry of Statistics and Programme Implementation (MoSPI) publishes the Consumer Price Index (CPI) every month. This index tells policymakers, economists, and the Reserve Bank of India whether inflation is rising or falling. Within the CPI, there is a sub-component specifically for 'Transport by Air' -- i.e., airfare inflation.

The critical problem is that MoSPI's airfare data is collected manually by surveyors visiting travel agents and booking counters. This process takes 30-45 days. By the time the government publishes 'June airfare CPI', we are already in August, and fares may have completely changed due to fuel price movements, monsoon seasons, or new airline capacity.

Our prototype solves this by building an automated, real-time data pipeline that:

- Scrapes live flight prices from Google Flights and airline APIs every few hours
- Cleans the raw data by removing duplicates and statistically impossible prices
- Computes an Airfare CPI using the same Laspeyres Index formula as India's official CPI
- Detects unusual price spikes automatically and flags them for human review
- Displays everything on a live web dashboard accessible from any browser
- Runs entirely autonomously in the cloud, 5 times per day, without any human intervention

The result is an airfare inflation signal that is available TODAY -- not 30 days later. This gives policymakers, researchers, and the airline industry a real-time early warning system for inflationary pressure in domestic air travel.

2. Tools Used -- Detailed Explanation of Each

Every tool was chosen deliberately. Below is an honest, technical explanation of what each tool does, why it was chosen over alternatives, and what specific problem it solves inside the prototype.

2.1 Python 3.11 -- The Core Language

Python is the programming language the entire system is written in. Every script, from the web scraper to the CPI calculator to the web dashboard, is Python.

Why Python and not Java or Node.js?

Python dominates data science and automation for a concrete reason: its ecosystem. Libraries like Pandas (data manipulation), scikit-learn (machine learning), Plotly (interactive charts), and Streamlit (web apps) do not have equivalents of the same quality in other languages. What would take 500 lines in Java takes 20 lines in Python using these libraries. For a hackathon where speed matters, this is the decisive factor.

Why version 3.11 specifically?

Python 3.11 introduced significant performance improvements (10-60% faster than 3.10) and better error messages. It is also the version supported by GitHub Actions' ubuntu-latest runner and Streamlit Community Cloud, ensuring identical behavior locally and in the cloud.

2.2 fast-flights -- Google Flights Price Scraper

This is the most technically interesting component of the entire system. fast-flights is a Python library that intercepts and decodes Google Flights' internal binary data protocol (protobuf format) to extract real flight prices.

How does it actually work?

When you visit Google Flights in a browser and search for a flight, your browser sends a request to Google's servers. Google responds with flight data encoded in a binary format called Protocol Buffers (protobuf) -- a compact binary serialization format developed by Google. A normal human cannot read this binary data. fast-flights knows the exact structure of this protobuf message. It constructs a valid search query, sends it directly to Google's servers, receives the binary response, and decodes it back into readable flight objects with prices, airlines, and times. No browser is opened. No page rendering happens. The entire operation completes in 1-2 seconds per route.

The critical problem it solved for us: currency

When our code runs on GitHub's servers in the United States, Google detects that the request is coming from a US IP address and returns prices in US Dollars (USD). Our initial code did not specify a currency, so prices like \$93 were being saved to our database as '93 INR'. This caused our CPI to be calculated on values that were 83x smaller than the real prices (since 1 USD = ~83 INR). The fix was to explicitly pass currency='INR' in the query, which forces Google to return all prices in Indian Rupees regardless of server location.

Why not scrape airline websites directly?

Direct airline website scraping requires opening a real browser (via Playwright or Selenium), navigating multiple pages, handling cookie banners, solving captchas, waiting for JavaScript to render, and reading DOM elements. This process takes 5-15 minutes per route and breaks every time the airline changes their website layout. fast-flights gives us the same underlying data in 2 seconds without any of these problems.

2.3 Pandas -- Data Processing and CPI Calculation

Pandas is Python's most widely used library for working with tabular data (rows and columns, like a spreadsheet or database table). It is the backbone of our cleaning, outlier detection, and CPI calculation logic.

What it does in our system

- Loads thousands of raw price rows from the Supabase database into memory as a DataFrame (table)
- Groups rows by route (origin+destination) and date for statistical analysis
- Computes IQR-based outlier bounds per group ($Q1 - 1.5 \cdot IQR$ to $Q3 + 1.5 \cdot IQR$)
- Detects duplicate rows using a multi-column key (source, route, airline, hour)
- Calculates rolling averages and standard deviations for anomaly detection (Z-score)
- Computes the Laspeyres CPI formula across all routes in a single vectorised operation

Why not use raw SQL for everything?

SQL is excellent for filtering and joining data from a database, but it is very limited for statistical operations like rolling windows, IQR calculation, and complex groupby transformations. Pandas handles these operations natively and expressively in just a few lines of code. The pattern in our system is: use SQL to pull data OUT of the database, then use Pandas to do the statistical heavy lifting in Python memory.

2.4 Supabase and PostgreSQL -- The Cloud Database

PostgreSQL is one of the world's most advanced open-source relational databases. Supabase is a cloud service that hosts a fully managed PostgreSQL instance for free, providing a permanent, cloud-accessible database without needing to manage servers.

Why a cloud database and not a local CSV file?

A CSV file lives on one specific computer. If the laptop is off, the file cannot be accessed. If two processes write to a CSV simultaneously, the file becomes corrupted. A cloud database like Supabase solves all of these problems: it is accessible from anywhere in the world (your laptop, GitHub's US servers, Streamlit's cloud dashboard), it handles simultaneous writes from multiple sources safely using transactions, and it never goes offline because Supabase manages the infrastructure.

Key database tables in our system

Table Name	What It Stores
fare_observations	Every raw price scraped. Never modified or deleted.
fare_observations_clean	Same data but with is_duplicate and is_outlier flags added by cleaning.py.
airfare_index	The computed CPI values per route, written by index_calc.py after each pipeline run.
official_cpi	Official MoSPI benchmark data (loaded from government Excel files).

Transaction Pooler -- why the connection string looks unusual

Supabase provides a 'Transaction Pooler' (port 6543) instead of a direct database connection (port 5432). The pooler sits between our code and the database and manages a pool of reusable connections. This is critical for cloud deployments: without pooling, each GitHub Actions run would open a new raw TCP connection to the database, and Supabase's free tier would hit its connection limit after just a few parallel scrapes. The pooler recycles connections efficiently.

2.5 Streamlit -- The Web Dashboard

Streamlit is a Python library that converts a plain Python script into a fully interactive web application. You write Python code with `st.metric()`, `st.line_chart()`, `st.dataframe()` etc., and Streamlit automatically renders them as UI components in the browser -- with no HTML, CSS, or JavaScript required.

Why Streamlit over Grafana, Flask, or React?

Grafana requires a dedicated server, complex data-source configuration files, and a JSON-based dashboard provisioning system -- typically 1-2 days of setup. Flask or React require writing frontend HTML/CSS/JS code separately from the Python logic. Streamlit eliminates all of this: the entire dashboard is written in one Python file (`dashboard.py`) with the data logic and the UI in the same place. For a hackathon, this difference is the difference between having a live demo and not having one.

How Streamlit connects to the database

On every page load or refresh, `dashboard.py` calls `get_engine()` which establishes an SQLAlchemy connection to Supabase. It then reads the latest data directly from the database tables using SQL queries. The `@st.cache_data(ttl=60)` decorator caches the query results for 60 seconds to avoid hammering the database on every single user interaction (scrolling, clicking dropdowns, etc.).

2.6 Plotly -- Interactive Charts and the India Map

Plotly is an open-source graphing library that generates interactive, browser-based charts. Unlike Matplotlib (which generates static image files), Plotly charts can be zoomed, panned, hovered over for tooltips, and filtered by clicking the legend -- all without reloading the page.

How the India route map works technically

The map uses Plotly's Scattermap class (new in Plotly 7.0, replacing the deprecated Scattermapbox). It renders two layers: (1) Line traces connecting airport coordinate pairs (latitude/longitude) to draw the route lines in glowing cyan color, and (2) A scatter trace with 'markers+text' mode that places a red dot and a bold city name label at each airport's geographic coordinates. The base map tiles come from OpenStreetMap, which is a free, open-source map dataset requiring no API key. The entire map is passed to Streamlit via `st.plotly_chart()` and becomes fully interactive -- users can zoom into individual airports, pan across India, and hover over route lines to see live fare data.

Why Plotly 7.0 broke our code and how we fixed it

Plotly 6.x used a class called Scattermapbox that required a Mapbox API key for the map tiles and used a layout property called 'mapbox' for configuration. Plotly 7.0 completely removed both of these and renamed them to Scattermap and 'map' respectively. When we upgraded Plotly, the dashboard crashed immediately with `AttributeError: module 'plotly.graph_objects' has no attribute 'Scattermapbox'`. The fix required rewriting the map rendering code and also switched us from Mapbox satellite tiles (which required an account and API key) to OpenStreetMap tiles (free, no key needed, and actually much more readable for an India city map).

2.7 GitHub Actions -- Cloud Automation

GitHub Actions is a CI/CD (Continuous Integration and Continuous Deployment) platform built into GitHub. It allows you to define automated workflows in YAML files that run on GitHub's cloud servers in response to events -- including scheduled times (cron triggers).

How it works in our system

The file `.github/workflows/smart_scraper.yml` defines a workflow called 'AirPrice India Smart Scraper'. GitHub reads this file and automatically provisions an Ubuntu 22.04 virtual machine on its servers at each scheduled time. The VM runs our full pipeline (install dependencies, start mock API, run collectors, clean, compute index), then shuts down and is destroyed. The entire VM lifecycle takes about 4-6 minutes per run. Because GitHub provides this for free on public repositories, our pipeline runs in the cloud 5 times per day at zero cost.

The 30-minute early scheduling problem

GitHub's free-tier cron uses a shared queue. At peak hours (especially UTC 00:00, 06:00, 12:00, 18:00 when many developers schedule their jobs), thousands of workflows are queued simultaneously. GitHub processes them first-come, first-served, and a job scheduled for 00:00 UTC may not actually start until 00:15 or 00:25. To guarantee our data is ready by the target IST hour, we schedule 30 minutes early: our 7 PM IST target runs at 6:30 PM IST (13:00 UTC) so even with a 20-minute queue delay, the pipeline finishes before 7 PM.

Secrets management -- how the database password stays safe

The Supabase database password cannot be written directly in the YAML file because the GitHub repository is visible to teammates. GitHub provides an encrypted 'Secrets' vault (Settings -> Secrets and Variables -> Actions). We store the full `DATABASE_URL` string there as a secret named `DATABASE_URL`. In the YAML, we reference it as `${{ secrets.DATABASE_URL }}`, and GitHub injects it as an environment variable into the workflow at runtime. The actual password is never visible in the repository, logs, or build output.

2.8 schedule (Python Library) -- Local Automation

The 'schedule' library is a simple Python cron alternative. Instead of requiring the operating system's cron daemon (which is complex to configure on Windows), it runs a scheduling loop entirely within a Python process.

How smart_scheduler.py works

`smart_scheduler.py` registers five daily jobs (one for each target time) and then enters an infinite loop: check if any job is due, run it if yes, sleep 1 second, repeat. When the script starts, it also immediately runs the pipeline once as a 'catch-up' scrape -- this handles the case where the PC was shut down during a normally-scheduled time and missed a run. The script runs as a background daemon process using PowerShell's `Activate.ps1` to first activate the virtual environment.

The daemon restart problem

A Python process loads its source code entirely into memory at startup. If you edit `smart_scheduler.py` while it is already running (e.g., to add a new time), the running process does not notice the file changed. It keeps running the OLD code with the OLD schedule times. This is why adding 8 PM and 11 PM to the schedule had no effect until we killed and restarted the process. The fix is simple but non-obvious: any time the scheduler code changes, always restart the running daemon.

2.9 SQLAlchemy -- Database Connection Manager

SQLAlchemy is Python's most widely used database toolkit. It provides two things: (1) a connection engine that manages the low-level TCP socket connection to PostgreSQL, and (2) a query interface that lets us write database operations in Python while it handles SQL dialect differences.

pool_pre_ping -- the silent reliability fix

In our `db.py`, we create the engine with `pool_pre_ping=True`. This is a critical reliability setting. Database connections over the internet can be silently dropped by routers or firewalls after periods of inactivity (this is

especially common with Supabase's connection pooler, which has idle timeouts). Without `pre_ping`, our code would try to run a query on a dead connection and crash with a confusing `OperationalError`. With `pre_ping=True`, SQLAlchemy sends a lightweight `SELECT 1` test query before every real query. If the connection is dead, it transparently establishes a new one before proceeding. The user sees nothing; the query just works.

2.10 tenacity -- Automatic Retry with Exponential Backoff

Network requests -- especially to external services like Google Flights -- can fail unpredictably due to temporary server overload, rate limiting, or transient network errors. `tenacity` provides a retry decorator that automatically retries failed function calls.

Exponential backoff -- why not just retry immediately?

If Google's server is overloaded and we retry immediately, we are contributing to the overload. Exponential backoff means we wait progressively longer between retries: first wait 20 seconds, then 40 seconds, then 80 seconds. This gives the server time to recover and dramatically reduces the chance of getting permanently blocked or rate-limited. Our config (`MAX_RETRIES=3`, `BACKOFF_BASE_SECONDS=20`) means a worst-case total wait of $20+40+80=140$ seconds before giving up -- acceptable for a background automation task.

2.11 scikit-learn -- Machine Learning for Anomaly Detection

`scikit-learn` is Python's standard machine learning library. In our system it is used optionally for the Isolation Forest anomaly detection algorithm, switchable via the `METHOD` variable in `anomaly.py`.

What is Isolation Forest and how does it work?

Isolation Forest works on a clever insight: anomalies are easier to isolate than normal data points. The algorithm builds many random decision trees. For each data point, it measures how many splits are needed to isolate that point in its own tree leaf. Normal prices (which cluster together) require many splits to isolate. Anomalous prices (which are far from the cluster) are isolated in very few splits. Points with low average path length get a high anomaly score. We set `contamination=0.05`, meaning we expect about 5% of prices to be anomalous. The algorithm is well-suited for detecting price spikes that are far above or below the normal cluster, regardless of the specific threshold.

Why Z-Score is the default instead of Isolation Forest

Isolation Forest needs at least 50-100 data points per route to have enough variation to learn what 'normal' looks like. In the early days of the prototype (first week of scraping), each route may only have 10-20 observations. Training Isolation Forest on 10 points causes it to either flag everything or nothing -- both useless. Z-Score works meaningfully from as few as 3-4 observations (`min_periods=3` in our rolling window). This is why Z-Score is the default and Isolation Forest is the documented upgrade path for when the dataset matures.

2.12 python-dotenv -- Secrets Management for Local Development

`python-dotenv` reads a `.env` file from the project directory and loads its contents as environment variables, making them available to the Python process via `os.environ`. The `.env` file contains our Supabase `DATABASE_URL` with the password.

Why not just hardcode the password in `config.py`?

A hardcoded password in `config.py` would be committed to the GitHub repository and become visible to anyone with repository access. Even in a private repo, this is a bad practice: if the repo is ever accidentally made public or cloned to another machine, the password is exposed. With `python-dotenv`, the `.env` file is listed in `.gitignore` so it is never uploaded to GitHub. Each developer runs with their own local `.env` file. GitHub Actions uses its own secrets vault. In both environments, the code reads the password the same way: `os.environ.get('DATABASE_URL')`, but the source of the value is different and safe.

3. Complete Workflow -- Every Step, Every Challenge, Every Solution

The following describes the complete journey of data from the moment a scrape is triggered to the moment it appears on the dashboard. Each stage includes the specific real-world challenges it faces and exactly how the code handles them.

Stage 1: Trigger -- How the Pipeline Starts

The pipeline is triggered in one of two ways, running in parallel:

- **LOCAL:** `smart_scheduler.py` runs as a background daemon on the developer's PC. At each scheduled time (03:55, 11:55, 18:55, 19:55, 22:55 IST), it calls `run_prototype.ps1` via `subprocess.run()`. The 5-minute-early trigger is intentional -- it gives the ~5 minute scraping process time to complete so data is in the database by the top of the hour.
- **CLOUD:** GitHub Actions cron triggers the `smart_scraper.yml` workflow on an Ubuntu server. Triggered 30 minutes early (e.g., 06:30 PM IST for 07:00 PM target) to compensate for GitHub's queue delays.

Challenge: What if the PC was off during a scheduled time?

If the laptop is shut down at 7 PM, the local 18:55 scrape is missed. The cloud GitHub Actions run would still have completed, but when the laptop boots up again, the local scheduler starts fresh with no knowledge of missed runs. Solution: `smart_scheduler.py` contains a 'catch-up' block that runs the full pipeline immediately on startup, before entering the schedule loop. This ensures there is always a fresh scrape after any PC restart, regardless of what time it is.

Stage 2: Data Collection -- Scraping Flight Prices

The `run_prototype.ps1` script calls three collectors in sequence:

collector_tier1.py (Tier 1 -- Google Flights): This is the primary source of real-world price data. For each of the 3 routes (DEL-BOM, BOM-BLR, DEL-BLR) and 4 booking windows (3, 7, 14, 30 days), it queries Google Flights via `fast-flights` and records the prices of all returned flights. Each query is separated by a random delay of 4-9 seconds (politeness throttle) to avoid triggering rate limits. With 3 routes x 4 windows = 12 queries at ~7 seconds average = approximately 84 seconds total for collection.

Challenge: Rate limiting and anti-bot detection

Google's servers monitor for automated traffic. If the same IP sends too many requests in quick succession with identical request headers, Google blocks it. We handle this with: (1) Random delays between requests (`MIN_DELAY_SECONDS=4`, `MAX_DELAY_SECONDS=9`) so the pattern is not perfectly periodic, (2) The `curl_cffi` library, which generates HTTP requests that mimic a real browser's TLS fingerprint rather than Python's default `urllib` fingerprint, and (3) Tenacity's automatic retry with exponential backoff for any failed request.

Challenge: Currency mismatch (the USD bug)

As described in Section 2.2, Google returns prices in the currency matching the server's geographic location. GitHub's servers are in the US, so without explicit currency specification, all prices come back in USD. With INR to USD being roughly 1:83, a flight priced at Rs 7,700 INR would be returned as \$93 USD, which would be saved as '93 INR' -- a catastrophically wrong value. We deleted 8,600 corrupt rows and added `currency='INR'` to every query.

collector_tier0_dgca.py (Tier 0 -- DGCA Government Data): In the final production system, this would fetch official Directorate General of Civil Aviation traffic data. In the prototype, it generates realistic mock data to represent government-sourced observations. This is labelled `source_tier='tier0_dgca'` in the database so it can be filtered separately.

collector_tier2_airline.py (Tier 2 -- Direct Airline APIs): In production, this would hit IndiGo, Air India, and Akasa Air's booking APIs directly. In the prototype, it hits a locally-running mock API server (`api.py`, running on port 8000 via Uvicorn) that returns realistic simulated prices. This validates the pipeline architecture without needing real airline API keys.

Stage 3: Data Cleaning -- Building a Trustworthy Dataset

Raw scraped data is inherently dirty. Multiple collections at different times produce duplicate records. Scraper bugs or website glitches produce impossible prices. `cleaning.py` processes all raw rows and produces the `fare_observations_clean` table.

Duplicate Detection

A duplicate is defined as two rows with the same `source_tier`, `origin`, `destination`, `departure_date`, `airline`, AND the same `hour_bucket` (the hour of the day when the observation was scraped). The `hour_bucket` is computed as the timestamp floored to the nearest hour -- e.g., 18:42 and 18:57 both become `hour_bucket=18:00`. This design is deliberate: if the scraper runs twice within the same hour (e.g., 18:55 local + 18:30 GitHub Actions), we keep only the first observation. If they run in different hours, both are kept because prices may have changed.

Outlier Detection using IQR

For each group of (`origin`, `destination`, `departure_date`), we compute the Interquartile Range (IQR) of prices. Q1 is the 25th percentile price (cheaper end), Q3 is the 75th percentile (expensive end), and $IQR = Q3 - Q1$. Any price below $(Q1 - 1.5 * IQR)$ or above $(Q3 + 1.5 * IQR)$ is flagged as an outlier. This method is robust to extreme values -- unlike standard deviation, IQR is not distorted by the outliers themselves.

Challenge: Groups too small for meaningful statistics

IQR calculation requires at least 4 data points to be meaningful. With fewer points, the IQR would be based on so few values that a completely normal price could be flagged as an outlier just because the sample is small. Solution: the `_flag_group()` function explicitly returns `False` (no outlier) for any group with fewer than 4 price observations. We would rather miss a real outlier in a small group than falsely exclude valid data.

Design principle: flag, never delete

All rows -- including duplicates and outliers -- are written to `fare_observations_clean` with their flags set. The raw table is also preserved unchanged. This means the team can always answer the audit question: 'Why was this fare excluded from the index?' by querying the clean table and reading the `is_duplicate` or `is_outlier` flag. There is no silent data loss anywhere in the system.

Stage 4: Index Calculation -- Computing the Airfare CPI

`index_calc.py` reads all rows from `fare_observations_clean` where `is_duplicate=FALSE` and `is_outlier=FALSE` and price IS NOT NULL. For each unique (`origin`, `destination`) route pair, it computes the CPI.

The Laspeyres Index Formula

$$\text{index_value} = 100.0 \times (\text{mean price across all clean observations}) / (\text{mean price on the earliest date})$$

The `base_date` is determined dynamically: for each route, it is the earliest date for which we have cleaned, non-outlier price data in the database. This is a 'living base period' that automatically adjusts to whenever the system first collected reliable data for that route.

Challenge: Base period with no data

It is possible for the `base_price` to be NaN (Not a Number) if the earliest date's data was entirely removed as outliers. Dividing by NaN or zero would produce an infinite or invalid CPI value. Solution: the code explicitly checks `pd.isna(base_price)` or `base_price == 0` and skips the route with a warning log rather than producing a garbage index value.

National CPI -- weighting routes together

The dashboard computes a national CPI by taking a passenger-weighted average of all route CPIs. The weight for each route is its observation count -- routes with more price observations (a proxy for higher passenger traffic) contribute more to the national number. This mimics how MoSPI weights different transport modes in the official CPI basket.

Stage 5: Anomaly Detection -- Catching Suspicious Price Spikes

`anomaly.py` is run after the index calculation as the final pipeline step. It reads the same clean data and looks for price observations that are statistically far from recent norms.

Z-Score Rolling Window Algorithm

For each route, prices are sorted by time. A rolling window of the last 10 observations is maintained. For each new price, the algorithm computes: $Z\text{-Score} = (\text{price} - \text{rolling_mean}) / \text{rolling_std}$. If $|Z\text{-Score}| > 2.5$, the price is considered anomalous and a WARNING is logged. The rolling nature means the 'normal' reference keeps updating as new prices come in -- the system adapts to gradual, seasonal price changes automatically.

Challenge: Festival spikes -- real inflation or anomaly?

This is the most important conceptual challenge in the system. If average fares on DEL-BOM are Rs 7,000 and they jump to Rs 15,000 for Diwali, the Z-Score will be very high and the anomaly detector will fire an alert. BUT -- we do NOT remove festival prices from the CPI. The anomaly detection is purely a human alert system, not an automatic deletion. A human reviewer investigates the alert and determines: is this a real festival spike (keep it in CPI, it represents real consumer burden) or is it a data error like a website glitch returning Rs 9,99,999 (exclude it)? The distinction between 'market anomaly' and 'data error' requires human judgment, and the system is designed to surface that judgment to a human rather than making the decision automatically.

Demo mode for the hackathon presentation

Running `anomaly.py --inject-test-anomaly` adds one synthetic row with a price 5x the current mean to an in-memory copy of the data (the real database is never modified). This guarantees the anomaly detector fires a visible ALERT in the logs during the demo, proving the detection mechanism works -- even if no real anomalies happen to exist on the demo day.

Stage 6: Dashboard -- Displaying Results to the World

`dashboard.py` is a Streamlit application deployed publicly on Streamlit Community Cloud. It reads all data live from Supabase on every page load and renders four sections.

Section 1: Interactive India Route Map

Built with Plotly Scattermap on OpenStreetMap tiles. Two trace layers: (1) Line traces between airport coordinate pairs for route visualization, (2) Marker+text traces at each airport for city labels. Hovering over any route line shows the current average fare for that route. The map is centered at lat=21, lon=78 (geographic center of India) at zoom level 3.8.

Section 2 and 3: Fare Trends and Data Table

A pivot table groups prices by booking_window_days and source_tier to show how prices change as the travel date approaches. A raw data table shows the 50 most recent scraped observations for transparency.

Section 4: The CPI Calculator

The most important section for the hackathon. It computes both route-level and national CPI live in the browser from the Supabase data. It shows the full formula worked out step-by-step (base date, base price, current price, result). The color-coded bar chart uses a Red-Yellow-Green reversed color scale centered at CPI=100: routes above 100 are progressively redder (inflation), routes below 100 are progressively greener (deflation). A white dashed horizontal line at CPI=100 serves as the visual baseline reference.

Challenge: The Streamlit app was not visible to teammates

After deploying on Streamlit Community Cloud with a private GitHub repository, the deployed app was automatically set to 'Private' visibility -- only the repository owner could access it. Teammates and judges received the error 'You do not have access to this app'. Solution: in the Streamlit app settings, change App Visibility from Private to Public. The GitHub code repository remains private; only the web URL becomes publicly accessible.

4. Recommended Scraping Times and Rationale

Airline pricing is dynamic -- prices change multiple times per day based on demand, competitor prices, seat availability, and algorithmic yield management. Scraping at the wrong times gives a distorted picture of actual market prices.

Time (IST)	Rationale
04:00 AM	Airlines run batch pricing recalculations overnight (typically 1 AM - 3 AM IST). By 4 AM, all algorithms have completed a
12:00 PM (Noon)	Mid-morning demand signals from the 9 AM - 11 AM booking peak cause airlines to reprice upward on high-demand rou
07:00 PM	The single most important scraping time. Consumer browsing peaks after work hours (6 PM - 9 PM). Airline dynamic pri
08:00 PM	Some airlines (especially IndiGo) run a secondary pricing update at 7:30-8:00 PM after observing the initial evening der
11:00 PM	Airlines sometimes reduce prices on unsold seats in the final hour before midnight. Their yield management systems re

Times to avoid

- 2:00 AM - 3:30 AM IST: Airlines are mid-recalculation. Prices are transitional and inconsistent.
- 11:00 AM - 12:00 PM IST: Ticket booking systems perform maintenance windows. Some carriers have brief periods of elevated error rates during this window.
- Do not scrape more than once per hour from the same IP: increases rate-limit risk significantly.

Best single time if you can only scrape once per day: 7:00 PM IST. Peak demand, peak pricing, maximum representativeness of actual consumer experience.

5. Important Problems Encountered and How We Solved Them

Problem 1: USD vs INR Currency Bug [SEVERITY: CRITICAL]

What happened: All prices stored in the Supabase database were 2-digit numbers (e.g., 93, 89, 100 INR) instead of the expected thousands of rupees. The CPI values were computing as 100 +/- 2 points when in reality there was meaningful inflation. The entire index was meaningless.

Root Cause: The fast-flights library queries Google Flights without specifying a preferred currency. Google detects the geographic location of the requesting IP address and returns prices in the local currency. GitHub Actions runs on servers in the United States, so Google returned prices in USD (e.g., \$93). Our code stored this as '93 INR', making a Rs 7,700 ticket appear to cost Rs 93. The bug was invisible until we manually looked at the database rows -- the dashboard was not showing obvious errors because the relative CPI calculation still produced numbers near 100 (since all prices were wrong by the same factor, the ratio was preserved).

Fix Applied: Added `currency='INR'` as an explicit parameter to `create_query()` in `collector_tier1.py`. Google now returns INR prices regardless of server location. Deleted 8,600 corrupt rows from both `fare_observations` and `fare_observations_clean` using a SQL `DELETE WHERE price < 500`.

Lesson: Always explicitly specify every parameter that has a geographic default when code runs on cloud servers. Currency, timezone, and locale settings are all vulnerable to this class of bug.

Problem 2: Plotly 7.0 Breaking Change [SEVERITY: HIGH]

What happened: After upgrading the Python environment, the dashboard refused to start. It crashed immediately with: `AttributeError: module 'plotly.graph_objects' has no attribute 'Scattermapbox'`.

Root Cause: Plotly released version 7.0 which removed the `Scattermapbox` class entirely (not just deprecated -- completely removed). All code written for Plotly 5.x and 6.x using `Scattermapbox` was broken. The old mapbox layout key was also removed. Both were renamed to `Scattermap` and `map` respectively in the new API.

Fix Applied: Rewrote all map rendering code to use `go.Scattermap` instead of `go.Scattermapbox`, and updated the layout to use `map=dict(...)` instead of `mapbox=dict(...)`. Also switched from Mapbox satellite tiles (which required a Mapbox API account and API key) to OpenStreetMap tiles (completely free, no account, actually more readable for an India city map with visible city names and roads).

Lesson: Pin major library versions in `requirements.txt` using `==` (exact match) rather than `>=` (minimum version) for visualization libraries. Breaking changes in major versions are common and can silently break deployed applications.

Problem 3: Scheduler Daemon Not Picking Up New Schedule Times [SEVERITY: MEDIUM]

What happened: After adding 8:00 PM and 11:00 PM to the schedule in `smart_scheduler.py`, those times never triggered. The 8 PM run simply did not happen.

Root Cause: Python processes load their source code into memory at startup and then run from that in-memory copy. The file on disk can change freely without affecting the already-running process. The old `smart_scheduler.py` process was still running in the background with the old schedule (only 3 times per day). It had no mechanism to detect that its source file had changed on disk.

Fix Applied: Killed the running scheduler process and restarted it to load the new code. Added this as a documented operational procedure: any time `smart_scheduler.py` is modified, kill and restart the daemon.

Lesson: Any long-running daemon process requires a restart to pick up code changes. Consider adding a file watcher (watchdog library) in future versions to auto-restart on code changes.

Problem 4: GitHub Actions Cron Queue Delays [SEVERITY: MEDIUM]

What happened: The 8:00 PM scrape was consistently arriving at 8:22 PM. Dashboard users checking at exactly 8 PM would see the previous run's data, not the fresh data.

Root Cause: GitHub Actions free-tier uses a shared cron queue. At popular cron times (especially whole hours like 14:30 UTC = 8:00 PM IST), thousands of developers' workflows are queued simultaneously. GitHub's infrastructure processes them first-come-first-served with limited parallelism on the free tier. The result is that a job scheduled for 14:30 UTC may not start until 14:45-14:50 UTC.

Fix Applied: Moved all GitHub Actions cron times 30 minutes earlier. The 8 PM IST target now starts at 7:30 PM IST (14:00 UTC). Even with a 20-minute queue delay, the pipeline finishes well before 8:00 PM.

Lesson: GitHub Actions free-tier cron is not suitable for exact-second or exact-minute scheduling requirements. Always build in a 20-30 minute buffer. For exact-time requirements, use a dedicated cloud VM (AWS EC2, Google Cloud Compute) with its own cron daemon.

Problem 5: Deployed Streamlit App Not Accessible to Teammates [SEVERITY: LOW]

What happened: After successfully deploying the dashboard to Streamlit Community Cloud, the team tried to share the link with a teammate. She received the error: 'You do not have access to this app or it does not exist. Please sign in to continue.'

Root Cause: When a Streamlit app is deployed from a private GitHub repository, Streamlit Community Cloud automatically sets the app's visibility to 'Private' to match the repository's privacy setting. Private apps require the viewer to be signed into Streamlit with a GitHub account that has been explicitly granted access to the app.

Fix Applied: In the Streamlit Cloud dashboard, navigated to App Settings -> Sharing -> App Visibility and changed it from 'Private' to 'Public'. The GitHub repository code remains private (no one can read the Python source code). Only the running web application URL becomes publicly accessible.

Lesson: After any cloud deployment, always test the link from an incognito browser window (or a different device not logged into any account) to verify public accessibility before sharing with judges or stakeholders.